IncuBrix 2027 — PS4 Video Demonstration Script

UNEDITED VIDEO • < 8 MINS

Long-Tail Destination Network • Single Source of Truth for Video Recording

CANDIDATE ROLE	TARGET ASSESSMENT LEVEL	TARGET VIDEO DURATION	RECORDING RULE
Associate Tech Partner	Outstanding (40/40 Marks)	7m 30s to 7m 45s	100% Continuous (Single Take)

Pre-Recording Setup & Verification Checklist

- [ ] Terminal working directory: `/Users/tectonic/Desktop/Incubrix/PS4`
- [ ] Python venv activated: `source .venv/bin/activate`
- [ ] API server running on port 8001: `http://localhost:8001`
- [ ] Browser open to Command Center UI: `http://localhost:8001/`
- [ ] Terminal font set to 15pt+ for high-definition readability
- [ ] Microphone tested; background noise and notifications silenced

Minute-by-Minute Walkthrough & Spoken Narrative

1. Architectural Overview & Assessment Scope

`0:00 - 1:00 (60s)`

VISUAL ON-SCREEN: Browser showing **Command Center UI** (`http://localhost:8001/`) and **README.md** Mermaid diagram.

> EXACT SPOKEN SCRIPT:
> "Hello, this is my submission demonstration for IncuBrix 2027 Problem Statement 4: Long-Tail Destination Network.
>
> Our mission is to autonomously distribute content to the fragmented long-tail internet—developer forums like Discourse, federated nodes like Lemmy, blogging networks like Dev.to and Hashnode, CMS engines like Ghost, and custom webhooks—while safeguarding community goodwill and strictly adhering to the 10-point IncuBrix Integration Standard.
>
> We have achieved all Must-Haves, Core Level, Advanced Level, and Outstanding Level criteria across 9 connectors, a pre-publish brand safety rules engine, intelligent content tailoring, and an empirical recommendation engine evaluated on ground truth."

2. Health Checks, OpenAPI 3.1 & Command Center UI

`1:00 - 2:00 (60s)`

VISUAL ON-SCREEN: Split-screen showing Terminal and Browser.

```
# Integration Standard Point 1: Health & Readiness checks
curl -s http://localhost:8001/health | jq .
curl -s http://localhost:8001/ready | jq .
```

EXACT SPOKEN SCRIPT:

"Here we see the service running locally with sub-millisecond response times. Both `/health` and `/ready` verify that our SQLite WAL database and storage providers are healthy, satisfying Integration Standard Point 1.

The complete API contract is documented in OpenAPI 3.1 format at `openapi.yaml`. On the right, our Command Center Web UI exposes live telemetry: 155 real long-tail destinations, 9 active connectors, and runtime operational kill-switches."

3. Automated Test Suite & Hard Cases (42 Tests)

`2:00 - 3:30 (90s)`

VISUAL ON-SCREEN: Terminal executing the test runner.

```
./run_tests.sh
# Or run directly: DATABASE_URL=sqlite:///./test_clean.db .venv/bin/pytest -v
```

EXACT SPOKEN SCRIPT:

"Now, let's run the automated verification suite. With a single command, pytest runs all 42 tests in under 1 second.

Notice the breadth of test coverage: we rigorously verify the Appendix A connector contract, 4-step certification, 1,000-row catalogue import, content tailoring, Precision@10 recommendation, and all 10 Integration Standards.

Crucially, we test 7 Hard Cases including publish timeouts, platform 429 rate limits, 401 token expirations, 422 payload rejections, oversized media handling, schema drift with automatic demotion to L0, and dirty CSV deduplication."

4. Async 202 Publishing, Idempotency & Assisted Packs

3:30 - 5:00 (90s)

VISUAL ON-SCREEN: Terminal running async publish cURL and polling job status.

```
# 1. Asynchronous publishing with Idempotency-Key header
curl -X POST http://localhost:8001/api/v1/publish   -H "Content-Type: application/json" -H "Idempotency-
Key: demo-key-001"   -d '{"destination_id": "discourse-dev-forum", "rendition": {"title": "Scaling
Distributed Pipelines", "body": "Deep dive into job orchestration.", "tags": ["python", "architecture"]}}'
| jq .

# 2. Re-send identical request to demonstrate idempotency
curl -X POST http://localhost:8001/api/v1/publish   -H "Content-Type: application/json" -H "Idempotency-
Key: demo-key-001"   -d '{ ... }' | jq .

# 3. Poll job status
curl -s http://localhost:8001/api/v1/jobs/<JOB_ID> | jq .
```

EXACT SPOKEN SCRIPT:

"Here is our core publishing lifecycle: POST `/api/v1/publish` returns an immediate HTTP 202 Accepted with a unique `job_id`, meeting Integration Standard Point 3.

When we re-send the exact same payload with the same `Idempotency-Key`, the service detects the duplicate and returns the existing cached job without double-posting, satisfying Point 4.

Polling the job status reveals progress transitions from 25% to 100%, returning the confirmed live post URL and signed output manifest. For platforms without public APIs, our Assisted Pack connector automatically packages Markdown, social copy, and media assets into a signed ZIP bundle."

5. Pre-Publish Brand Safety Rules & Ban Risk Meter

5:00 - 6:15 (75s)

VISUAL ON-SCREEN: Browser in **Policy Sandbox** tab or Terminal triggering intentional violations.

```
# Trigger frequency cap violation
curl -X POST http://localhost:8001/api/v1/publish   -H "Content-Type: application/json"   -d
'{"destination_id": "discourse-freq-cap-test", "rendition": {"title": "Duplicate Post", "body": "Spamming
another post within cooldown.", "tags": ["test"]}}' | jq .
```

EXACT SPOKEN SCRIPT:

"Community goodwill is fragile. When we attempt to publish to a forum that enforces a cooldown policy—such as 1 post per 7 days—our Rules Engine intercepts the request before touching the network.

It emits an explicit error: `DEST_RULE_FREQUENCY_CAP` with HTTP 422, clear retry guidance, and the exact timestamp when the destination becomes eligible again.

The same protection blocks aggressive promotional posts with `DEST_SPEC_VIOLATION`, strips affiliate tracking links, and enforces AI disclosure notices. In our Command Center Studio, our Ban Risk Meter continuously calculates marketing intensity and assigns an objective safety grade."

6. Catalogue Import Benchmark & Recommendation Engine

6:15 - 7:15 (60s)

VISUAL ON-SCREEN: Terminal running the 1,000-row benchmark script and recommendation cURL.

```
# 1. Benchmark 1,000 synthetic destinations import
.venv/bin/python -c "
import asyncio
from src.engine.catalogue_importer import import_catalogue_csv
with open('data/synthetic_1000_destinations.csv', 'rb') as f:
    res = asyncio.run(import_catalogue_csv(f.read()))
    print(f'Imported {res["imported"]} rows in {res["duration_seconds"]}s with {len(res["errors"])}
errors')
"

# 2. Recommendation query
curl -X POST http://localhost:8001/api/v1/recommend   -H "Content-Type: application/json"   -d
'{"rendition": {"title": "CUDA GPU Video Pipelines", "body": "Hardware transcode benchmarks.", "tags":
["video", "gpu", "ai"]}}' | jq .
```

EXACT SPOKEN SCRIPT:

"For high-throughput catalogue operations, our CSV importer parsed, normalized, and deduplicated 1,000 destination rows in just 0.78 seconds—hundreds of times faster than the 5-minute requirement.

Our recommendation engine matches content against our researched 155-community catalogue, achieving 86.7% Precision@10 evaluated on our labeled ground-truth set, complete with human-readable rationale explanations."

7. Official Submission Form & Deliverables Wrap-Up

7:15 - 7:45 (30s)

VISUAL ON-SCREEN: Display `IncuBrix_Assessment_4_Distribution_Long_Tail_Destination_Network_Submission_Form.xlsx` in Excel or preview.

EXACT SPOKEN SCRIPT:

"Finally, here is our completed official submission workbook: `IncuBrix_Assessment_4_Distribution_Long_Tail_Destination_Network_Submission_Form.xlsx`. Every yellow cell, evidence link, metric, and declaration is completed.

The repository includes our OpenAPI 3.1 specification, full documentation in `README.md`, technical analysis in `REPORT.md`, licensing in `SOURCES.md`, AI disclosure in `AI_USE.md`, and all test pack outputs.

Thank you for your time and consideration."

Post-Recording Verification Checklist

- [ ] Video duration is strictly under 8 minutes (recorded between 7m 30s and 7m 45s).
- [ ] Terminal commands and browser UI text are razor-sharp and easily readable.
- [ ] Video is 100% continuous and unedited (single take, no cuts or splices).
- [ ] Uploaded to YouTube (Unlisted) or Google Drive (view permissions set to 'Anyone with link').